

目 录

1 第一部分 基于 face-api.js 的浏览器端人脸分析实验	1
1.1 实验背景与目标	1
1.2 项目结构与实验环境	1
1.2.1 项目结构	1
1.2.2 页面结构	2
1.3 核心技术与总体流程	2
1.3.1 face-api.js 模型调用	2
1.3.2 模型来源与训练机制	3
1.3.3 输入、推理与绘制流程	4
1.4 功能模块分析	5
1.4.1 图片表情识别	5
1.4.2 年龄与性别估计	5
1.4.3 摄像头实时追踪	6
1.5 实验结果与分析	7
1.6 存在问题与改进方向	7
1.7 本部分小结	8
2 第二部分 后续内容	9
2.1 待补充内容	9
参考文献	10
附录 人脸分析实验关键代码	11
页面入口与展示容器	11
图片表情识别脚本	12
年龄与性别估计脚本	16
摄像头实时追踪脚本	19
开源人脸识别模型训练代码	25

1 第一部分 基于 face-api.js 的浏览器端人脸分析实验

1.1 实验背景与目标

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型应用。传统的人脸分析系统通常把模型部署在服务器端，浏览器或客户端只负责采集图像、上传数据和展示结果。这种方式便于集中管理模型，但也带来了部署复杂、网络依赖较强、交互延迟较高等问题。随着 TensorFlow.js 等前端推理框架的发展，经过训练的深度学习模型可以直接在浏览器中加载与执行，使静态网页也能够完成较完整的人脸检测、关键点定位、表情识别和属性估计任务。

本实验基于 `code/face-recognition/` 目录中的纯前端人脸分析项目完成。该项目核心依赖为 `face-api.js[1]`，页面样式使用 Bootstrap 和 MDB，业务逻辑集中在 `dist/` 目录下的 JavaScript 脚本中。实验围绕三个实际页面展开：图片表情识别、图片年龄与性别估计、摄像头实时人脸追踪。通过阅读和运行这些页面，可以观察浏览器端人脸分析系统从模型加载、输入获取、模型推理到结果绘制的完整流程。

本部分实验目标如下：第一，分析静态网页项目中 HTML 页面、业务脚本、模型文件和示例资源之间的组织关系；第二，说明 `face-api.js` 在图片输入和摄像头视频流输入中的调用流程；第三，分别分析表情识别、年龄与性别估计、实时追踪三个模块的功能实现；第四，补充说明预训练模型的来源与人脸识别模型的训练思想；第五，总结该类前端人脸分析实验的效果、局限和改进方向。

1.2 项目结构与实验环境

1.2.1 项目结构

该项目没有后端服务和复杂构建入口，整体更接近一个可直接静态托管的演示站点。页面入口位于项目根目录，模型权重位于 `models/`，示例图片、标注人物图片和视频资源位于 `public/`，实际被页面引用的业务代码位于 `dist/`。

其中 `dist/face-api.min.js` 是 `face-api.js` 的浏览器端构建文件，其他脚本则分别对应不同实验功能。

主要页面、脚本和功能对应关系如表 1.1 所示。

模型目录中包含 Tiny Face Detector、SSD MobileNetV1、68 点人脸关键点、表情识别、年龄性别估计、人脸识别等预训练模型权重。脚本中的 均设置为，人物标注样本路径设置为 `./public/img/labeled_images`，说明当前版本已经把模型和示例资源改为从本地仓库加载。与完全依赖远程 GitHub Raw URL 或 CDN 的演示相比，

表 1.1 人脸分析实验页面与脚本对应关系

页面文件	业务脚本	主要功能
face_expression_recognition.html	dist/expression_script.js	图片表情识别
age_gender_estimation.html	dist/estimation_script.js	年龄与性别估计
webcam_face_tracking.html	dist/webcam_script.js	摄像头实时追踪

本地资源加载能够减少外部网络波动对实验的影响，也更适合课程报告中的复现实验。

1.2.2 页面结构

三个实验页面都采用相似的界面组织方式：页面顶部为导航栏，中间主体为左右分栏布局。左侧通常放置上传控件、摄像头状态或识别结果卡片，右侧为图像或视频展示区域。页面通过 `script` 标签延迟加载 `dist/face-api.min.js` 和当前功能对应的业务脚本，HTML 本身主要负责结构和交互容器，模型加载、推理和绘制逻辑均由 JavaScript 完成。

这种组织方式的优点是职责清晰。HTML 页面只描述用户界面，CSS 负责统一视觉样式，业务脚本负责把输入数据送入模型并把结果写回页面。由于没有后端服务，实验运行时只需要浏览器能够访问项目静态文件；对于摄像头功能，还需要浏览器支持媒体采集接口并允许页面获取摄像头权限 [5]。

1.3 核心技术与总体流程

1.3.1 face-api.js 模型调用

`face-api.js` 是建立在 `TensorFlow.js` 之上的 JavaScript 人脸分析库，封装了常见的人脸检测、关键点定位、人脸描述子、表情识别以及年龄性别估计接口。项目中的脚本通过 加载对应网络，并使用 并行等待模型加载完成。模型加载完成后，脚本才会注册图片上传监听器或启动摄像头连接流程。

不同实验模块加载的网络略有差异。图片表情识别页面加载 Tiny Face Detector、68 点关键点、人脸描述子、表情识别和 SSD MobileNetV1；年龄与性别估计页面加载 Tiny Face Detector、68 点关键点、人脸描述子、年龄性别估计和 SSD MobileNetV1；摄像头追踪页面加载 Tiny Face Detector、68 点关键点、人脸描述子和表情识别模型。虽然部分页面加载了当前流程中未显式使用的网络，但整体仍体现了人脸分析任务的典型链式调用方式。

1.3.2 模型来源与训练机制

需要区分的是，本项目中的 `models/` 目录保存的是 `face-api.js` 发布的预训练权重副本，而不是当前课程项目从零训练得到的新模型。页面运行时只通过 把这些权重加载到浏览器端执行推理；本地保存权重的作用是减少外部网络依赖，使实验可以稳定复现。换言之，本实验分析的是预训练模型的加载、调用和结果解释，而不是完整训练出 Tiny Face Detector、表情识别或年龄性别估计网络的过程。

`face-api.js` 的模型体系可以理解为多个已经训练好的子网络组合 [2]。人脸检测模型负责输出人脸候选框，关键点模型负责定位面部结构，人脸识别模型输出固定长度的人脸描述子，表情与年龄性别模型则在检测到的人脸区域上进行属性预测。其中，人脸识别模型的训练思想与 `dlib` 深度度量学习示例一致 [4]：网络把输入人脸图像映射为 128 维向量，使同一身份的图像在向量空间中距离更近，不同身份的图像距离更远。训练完成后，系统不需要直接输出“某个人”的类别，而是比较向量距离来判断两张人脸是否相似。

`dlib` 的开源示例 `dnn_metric_learning_on_images_ex.cpp` 展示了这种训练流程 [3]。数据按身份目录组织，每个子目录包含同一人的多张图片；训练时 `mini-batch` 同时采样多个人和每人的多张图片，使损失函数能够同时看到正样本对和负样本对。网络主体采用 ResNet 结构，输出层为 128 维向量，损失层使用 约束嵌入距离。训练循环不断取出 `mini-batch` 调用，学习率降低到阈值后将网络序列化保存为模型文件。

项目中的人物识别样本 `public/img/abeled_images` 也容易被误解为“训练集”。实际上，业务脚本只是读取这些样本图片，通过已加载的人脸识别网络提取描述子，再构造 和 。随后，新上传图片的人脸描述子会与这些参考描述子计算距离，距离低于阈值时显示对应人物名称。因此，这一步更接近“基于特征的样本登记与最近邻匹配”，并没有更新神经网络权重。

从数学表达上看，浏览器端人脸分析可以抽象为从输入图像到一组结构化人脸属性的映射。设输入图像为

$$I \in \mathbf{R}^{H \times W \times 3}, \quad (1.1)$$

其中 H 、 W 分别表示图像高度和宽度，3 表示 RGB 三个颜色通道。人脸检测网络首先在图像上预测候选框集合

$$\mathcal{B} = \{b_i = (x_i, y_i, w_i, h_i, s_i) \mid i = 1, 2, \dots, n\}, \quad (1.2)$$

其中 (x_i, y_i) 表示第 i 个检测框左上角坐标， w_i 和 h_i 表示宽度与高度， s_i 表示该框

为人脸的置信度。实际绘制时只保留置信度高于阈值 τ 的检测结果，即

$$\mathcal{B}' = \{b_i \mid s_i \geq \tau, b_i \in \mathcal{B}\}. \quad (1.3)$$

在每个检测框内，关键点定位模型继续估计 68 个面部关键点：

$$\mathcal{L}_i = \{(u_{ij}, v_{ij}) \mid j = 1, 2, \dots, 68\}. \quad (1.4)$$

这些关键点描述了眉毛、眼睛、鼻子、嘴唇和脸部轮廓等局部结构，为后续表情和属性估计提供更细粒度的几何信息。表情识别可以表示为对七类表情的概率分布预测：

$$\mathbf{p}_i = \text{softmax}(\mathbf{z}_i), \quad \hat{c}_i = \arg \max_k p_{ik}, \quad (1.5)$$

其中 $\mathbf{z}_i$ 为模型输出的类别得分， p_{ik} 表示第 i 张人脸属于第 k 类表情的概率， $\hat{c}_i$ 为最终显示的主表情。年龄与性别估计也可写成

$$(\hat{a}_i, \hat{g}_i, q_i) = f_{\text{attr}}(I, b_i, \mathcal{L}_i), \quad (1.6)$$

其中 $\hat{a}_i$ 表示估计年龄， $\hat{g}_i$ 表示性别标签， q_i 表示性别分类置信度。由此可见，项目中的链式 API 调用本质上对应了“检测框定位、关键点建模、属性分类或回归”的多阶段推理过程。

1.3.3 输入、推理与绘制流程

图片类页面的流程可以概括为以下五步。

第一步，监听 `input[type=file]` 的事件，获得用户上传的图片文件。第二步，通过 `FileReader` 将本地文件转换为浏览器中的图片对象，并将图片插入结果展示容器。第三步，使用 `getContext` 创建覆盖画布，再调用 `drawImage` 使画布尺寸与图片显示尺寸一致。第四步，调用人脸检测接口检测图片中的全部人脸，并根据具体任务继续串联关键点、描述子、表情或年龄性别估计接口。第五步，将检测结果通过 `drawText` 映射到展示尺寸，再通过 `drawImage` 系列接口绘制检测框、表情标签或自定义标注，同时把结构化结果渲染到侧边栏。

摄像头页面的流程与图片页面类似，但输入从静态图片变为连续视频帧。脚本先通过浏览器媒体采集接口请求摄像头视频流，并把视频流绑定到页面中的视频元素。当 `video` 触发 `onplay` 事件后，页面创建覆盖画布并启动定时器，每隔 100 ms 对当前视频帧执行一次人脸检测、关键点定位和表情识别。每轮推理结束后，脚本清空画布

并重绘最新结果，从而形成实时追踪效果。

1.4 功能模块分析

1.4.1 图片表情识别

图片表情识别由 `dist/expression_script.js` 实现。页面接收用户上传的 JPG、PNG 等常见图片格式，识别完成后在右侧图片上绘制人脸检测框和表情结果，并在左侧结果区域展示每张人脸的详细表情概率。

脚本预先定义了七类表情标签：中性、高兴、难过、生气、害怕、厌恶、惊讶。这些标签与 `face-api.js` 返回的 `neutral`、`happy`、`sad`、`angry`、`fearful`、`disgusted`、`surprised` 相对应。识别结果返回后，脚本按照检测框的横坐标对多张人脸排序，使页面展示顺序与图像中从左到右的人脸位置一致。随后，脚本遍历七类表情分数，选出概率最高的一类作为主表情，并用百分比和进度条展示每一类表情的模型置信度。

该模块的实现重点不只是调用表情模型，还包括结果解释和用户界面反馈。若没有检测到人脸，页面会显示“未检测到人脸”的空状态提示；若正在处理图片，结果区会先显示“正在识别”。这些状态反馈降低了用户对模型响应时间的困惑，也使实验页面更接近完整应用而非单纯的代码示例。

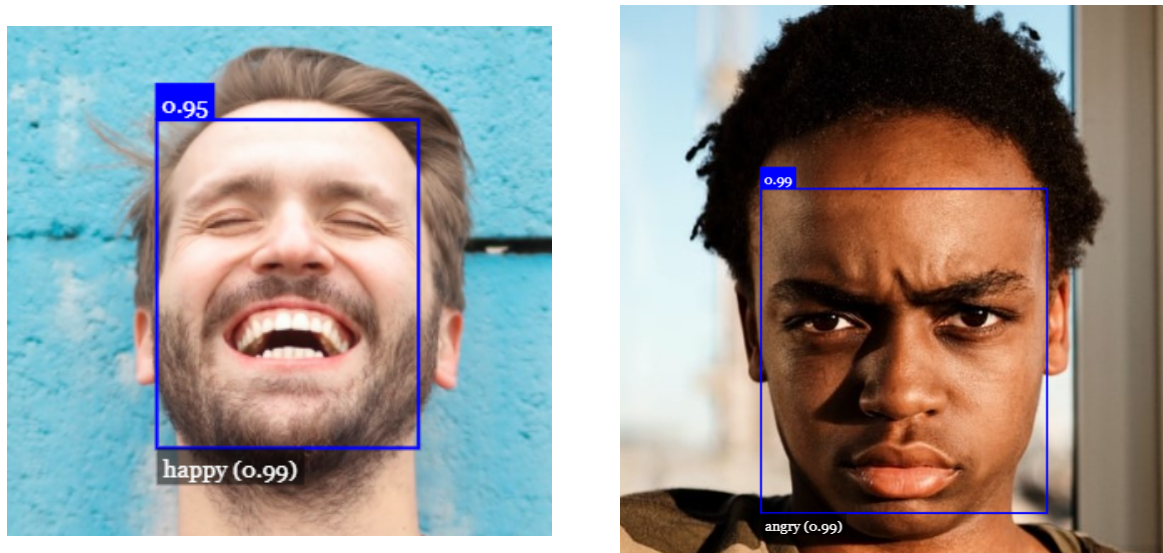


图 1.1 表情识别示例结果

1.4.2 年龄与性别估计

年龄与性别估计由 `dist/estimation_script.js` 实现。该模块与表情识别共用相似的图片上传和画布绘制流程，但推理链路改为在人脸检测之后继续执行关键点提取、人脸描述子提取以及年龄性别估计。模型返回的主要结果包括检测框、估计年

龄、性别标签和性别置信度。

在结果展示上，脚本将年龄四舍五入后以“岁”为单位显示，将 和 映射为“男性”和“女性”，并通过结果卡片展示每张人脸的年龄估计、性别判断和性别置信度。同时，脚本使用 在图片上叠加检测框标签，使用户能够把侧边栏中的结构化数据与图片中的具体人脸对应起来。

该模块反映了多任务人脸属性估计的基本思想：人脸检测首先确定候选区域，关键点与特征提取提供面部结构信息，属性估计网络再输出年龄和性别等外观属性。需要注意的是，年龄和性别估计受光照、姿态、遮挡、图像清晰度和训练数据分布影响明显，其结果应理解为模型预测值，而不是绝对真实标签。在实际系统中，这类输出尤其不适合直接用于高风险判断。

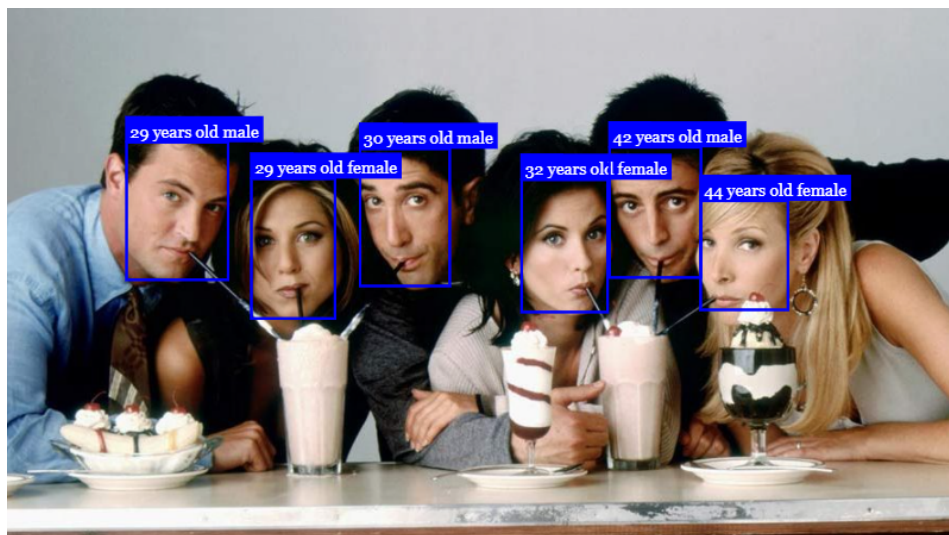


图 1.2 年龄与性别估计示例结果

1.4.3 摄像头实时追踪

摄像头实时追踪由 `dist/webcam_script.js` 实现。页面启动后首先显示“等待授权”状态，随后检查浏览器是否支持。如果浏览器支持该接口，脚本请求摄像头权限；如果用户拒绝授权或设备不可用，页面会显示访问失败提示。成功获取视频流后，页面等待视频播放事件，并开始周期性识别。

实时识别部分使用 以 100 ms 为间隔处理视频帧。每一轮识别都会调用 Tiny Face Detector 检测人脸，再串联关键点定位和表情识别。脚本随后根据当前展示区域尺寸调整画布，清除上一帧绘制内容，并重新绘制检测框、68 点关键点和表情标签。与图片页面相比，摄像头页面更强调连续性和响应速度，因此选用轻量级检测器更符合实时场景需求。

该模块还处理了若干工程细节。首先，会根据展示容器尺寸同步画布大小，窗口尺寸变化时也会重新匹配，避免视频画面与识别标注错位。其次，侧边栏结果设置了 280 ms 的刷新节流，并在结果内容没有变化时跳过重复渲染，降低高频检测对 DOM 更新的压力。最后，脚本监听视频暂停和结束事件，及时清除定时器并更新页面状态，避免后台持续执行无意义的推理任务。

1.5 实验结果与分析

从功能完整性看，实验系统完成了三个典型的人脸分析任务。表情识别页面能够在上传图片后检测多张人脸，并输出每张人脸的主表情及七类表情概率；年龄与性别估计页面能够输出检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在获得权限后持续处理视频帧，并实时叠加检测框、关键点和表情结果。

从工程实现看，该项目具有较好的演示性。项目采用静态资源组织方式，页面入口清晰，业务脚本集中，模型文件和示例数据保存在本地目录，便于离线讲解和复现实验。左侧结果区与右侧图像区域的布局也有利于对照观察模型输出：右侧用于判断人脸定位和标注位置是否正确，左侧用于查看更细粒度的概率、置信度和状态信息。

从性能角度看，图片类任务主要受模型加载时间和单次图片推理时间影响；摄像头任务还受到设备计算能力、摄像头分辨率、浏览器性能和当前页面负载影响。脚本采用 Tiny Face Detector 进行实时检测，并对侧边栏渲染做节流处理，说明项目已经考虑到实时场景中的性能压力。不过，如果输入视频分辨率较高或设备性能较弱，实时帧率仍可能下降。

从可靠性角度看，该系统的结果受数据质量影响较大。正脸、光照均匀、遮挡较少的人像通常更容易得到稳定结果；侧脸、模糊、强背光、多人脸重叠或表情不明显的图片可能导致漏检、误检或属性估计偏差。摄像头功能还依赖浏览器权限和运行上下文，某些浏览器或非安全环境可能无法直接访问摄像头。因此，本实验更适合作为前端深度学习推理和人脸分析流程的教学演示，而不应被理解为生产级身份识别系统。

1.6 存在问题与改进方向

当前项目已经能够展示浏览器端人脸分析的基本能力，但仍存在可以改进的地方。

第一，页面和脚本之间仍以全局 DOM 查询和普通脚本文件组织为主，随着功能增多，状态管理和代码复用会变得困难。后续可以把公共的模型加载、画布匹配、

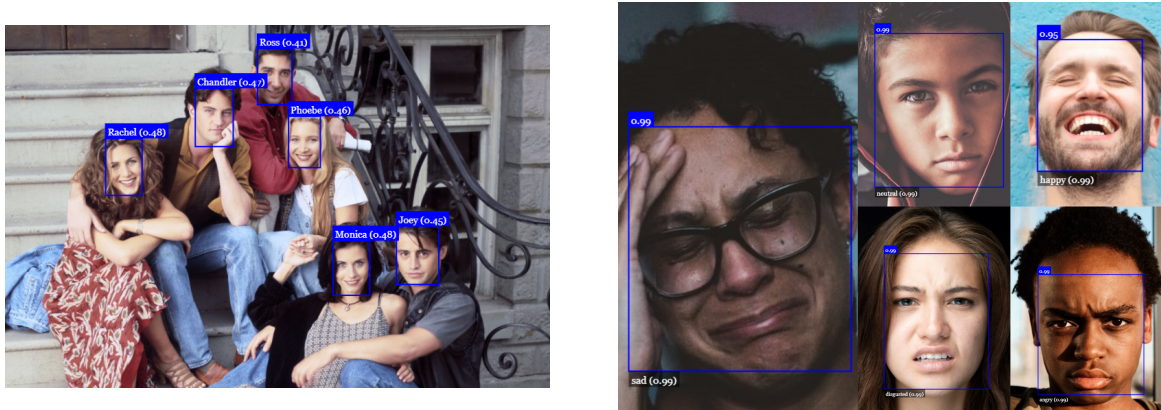


图 1.3 人脸检测与综合演示效果

空状态渲染、结果排序等逻辑抽取为共享模块。

第二，当前实验主要依赖预训练模型的直接输出，缺少定量评估。若要把实验写得更严谨，可以构造包含不同光照、姿态、遮挡和多人脸场景的测试集，统计检测成功率、属性估计稳定性和实时帧率等指标。

第三，摄像头功能涉及人脸和视频流等敏感信息。虽然当前项目在浏览器端本地处理数据，没有主动上传视频，但报告和界面仍应明确提示用户授权含义，并避免保存或传播未经同意的人脸图像。若继续扩展为实际应用，还需要加入更完整的隐私保护、权限说明和数据生命周期管理。

第四，年龄和性别估计属于外观属性推断，天然存在不确定性和潜在偏差。后续可以在界面和报告中弱化确定性表达，例如将结果称为“模型估计”而非“真实年龄”或“真实性别”，并在实验分析中说明其不适用于身份、资格、风险等高影响决策。

1.7 本部分小结

本部分围绕 `face-recognition` 项目完成了浏览器端人脸分析实验说明。实验展示了静态网页如何利用 `face-api.js` 加载本地模型，并在图片和摄像头视频流两类输入上完成人脸检测、关键点定位、表情识别以及年龄性别估计。通过分析三个页面的结构和脚本流程可以看出，前端深度学习推理适合课程演示、交互原型和轻量级体验场景，但在性能、鲁棒性、隐私保护和结果可信度方面仍需要进一步评估与完善。

2 第二部分 后续内容

本报告第二部分用于补充其他实验或扩展分析内容。当前先保留章节结构，后续可根据课程要求继续撰写研究背景、方法设计、实验过程、结果分析和改进方向。

2.1 待补充内容

本节暂不展开。后续写作时，建议与第一部分保持一致的结构：先说明问题背景和目标，再介绍系统结构或算法流程，最后给出实验结果、局限性和改进方案。

参考文献

- [1] Vincent Muhler. face-api.js: JavaScript API for face detection and face recognition in the browser and nodejs with tensorflow.js. GitHub, 2018.
- [2] Vincent Muhler. face-api.js model weights. GitHub repository.
- [3] Davis E. King. Dlib-ml: A Machine Learning Toolkit. Journal of Machine Learning Research, 2009.
- [4] Davis E. King. dlib C++ Library. GitHub repository.
- [5] World Wide Web Consortium. Media Capture and Streams. W3C Recommendation.
- [6] Bootstrap Team. Bootstrap Documentation.

附录 人脸分析实验关键代码

本附录列出浏览器端人脸分析实验中与功能实现直接相关的页面结构和业务脚本片段。为避免重复展示第三方库源码，附录不收录 `face-api.min.js`，只展示页面如何引入模型库、承载输入输出区域，以及三个实验模块如何完成模型加载、图片推理、结果渲染和摄像头实时追踪。

页面入口与展示容器

图片表情识别页面在文档头部延迟加载 `face-api.min.js` 和当前页面的业务脚本。主体区域左侧提供图片上传控件和识别结果容器，右侧的 `analysis-stage` 用于插入用户上传的图片和覆盖绘制结果的画布。

```
21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/expression_script.js"></script>

75 <section class="panel control-panel">
76   <h2 class="panel-heading"> 上传图片 </h2>
77   <input id="upload" type="file" accept="image/*" />
78   <label for="upload" class="upload-dropzone">
79     <span class="upload-label"> 选择图片 </span>
80     <span class="upload-hint"> 建议使用清晰、正脸较明显的照片。</span>
81   </label>
82   <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83 </section>
84
85 <section class="panel info-panel">
86   <h2 class="panel-heading"> 识别结果 </h2>
87   <div id="expression-results" class="result-summary">
88     <div class="result-empty">
89       <strong> 等待识别 </strong>
90       <span> 上传图片后，这里会同步显示表情识别结果副本。</span>
91     </div>
92   </div>
93 </section>
94 </aside>
95
96 <section class="panel result-panel">
97   <div id="analysis-stage" class="analysis-stage">
98     <div class="stage-empty" data-empty-state>
99       <strong> 等待图片 </strong>
```



```
100     <span> 上传后在这里显示表情识别结果。</span>
101   </div>
```

摄像头实时追踪页面同样通过脚本标签加载模型库和业务逻辑。页面中的追踪区域 `tracking-stage` 用来叠加视频与检测画布。

`video` 元素负责承接浏览器摄像头视频流，并为后续逐帧识别提供输入。

```
21   <script defer src="dist/face-api.min.js"></script>
22   <script defer src="dist/webcam_script.js"></script>
```

```
101   <section class="panel tracking-panel">
102     <div id="tracking-stage" class="tracking-stage">
103       <div class="stage-empty" data-empty-state>
104         <strong> 等待摄像头权限 </strong>
105         <span> 授权后在这里显示实时画面与识别结果。</span>
106       </div>
107       <video
108         id="video"
109         width="1280"
110         height="720"
111         autoplay
112         muted
113         playsinline
114       ></video>
115     </div>
```

图片表情识别脚本

表情识别脚本首先取得页面中的上传控件、展示区域和结果区域，并定义本地模型路径。随后通过 并行加载检测、关键点、人脸描述子、表情识别和 SSD MobileNetV1 模型，全部模型加载完成后才进入 流程。

```
1   const imageUpload = document.getElementById("upload");
2   const analysisStage = document.getElementById("analysis-stage");
3   const expressionResults = document.getElementById("expression-results");
4   const MODEL_URL = "./models";
5
6   const EXPRESSION_LABELS = {
7     neutral: " 中性",
8     happy: " 高兴",
```

```
9   sad: " 难过",
10  angry: " 生气",
11  fearful: " 害怕",
12  disgusted: " 厌恶",
13  surprised: " 惊讶",
14 };
15 const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
16
17 Promise.all([
18   faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
19   faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
20   faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
21   faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
22   faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
23 ]).then(start);
```

为了让模型输出更容易阅读，脚本将 `face-api.js` 返回的英文表情标签映射为中文标签，并按检测框横坐标排序多张人脸。结果渲染函数会选出概率最高的主表情，同时把七类表情概率渲染为进度条，便于观察模型对不同表情类别的置信度分布。

```
38 function getSortedDetections(detections) {
39   return [...detections].sort(
40     (first, second) => first.detection.box.x - second.detection.box.x
41   );
42 }
43
44 function getTopExpression(expressions) {
45   return EXPRESSION_ORDER.reduce(
46     (currentTop, key) => {
47       const score = expressions[key] || 0;
48       return score > currentTop.score ? { key, score } : currentTop;
49     },
50     { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
51   );
52 }
53
54 function renderExpressionResults(detections) {
55   const sortedDetections = getSortedDetections(detections);
56
57   if (!sortedDetections.length) {
58     renderEmptyState(
59       expressionResults,
```

```
60     " 未检测到人脸",
61     " 请尝试上传清晰、正脸更明显的照片。"
62 );
63 return;
64 }
65
66 expressionResults.innerHTML = sortedDetections
67   .map((detection, index) => {
68     const topExpression = getTopExpression(detection.expressions);
69     const metrics = EXPRESSION_ORDER.map((key) => {
70       const score = detection.expressions[key] || 0;
71
72       return `
73         <li class="result-metric">
74           <div class="result-metric__row">
75             <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
76             <span class="result-metric__value">${formatPercent(score)}</span>
77           </div>
78           <div class="result-bar"><span style="width: ${Math.round(
79             score * 100
80           )}%></span></div>
81         </li>
82       `;
83     }).join("");
84
85     return `
86       <article class="result-card">
87         <div class="result-card__header">
88           <span class="result-card__title"> 人脸 ${index + 1}</span>
89           <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
90         </div>
91         <p class="result-card__highlight">
92           主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
93           (${formatPercent(topExpression.score)})
94         </p>
95         <ul class="result-list">${metrics}</ul>
96       </article>
97     `;
98   }).join("");
99 }
100 }
```

图片上传监听器是该模块的核心入口。用户选择文件后，脚本先清理旧图片和旧画布，再将文件转换为浏览器图片对象，创建覆盖画布并匹配尺寸。随后，脚本依次执行人脸检测、关键点定位、人脸描述子提取和表情识别，最后将结果缩放到展示尺寸并绘制检测框和表情标签。

```
102 async function start() {
103   const container = document.createElement("div");
104   container.className = "analysis-stage__media";
105   analysisStage.append(container);
106
107   let image;
108   let canvas;
109
110   imageUpload.addEventListener("change", async () => {
111     if (!imageUpload.files.length) {
112       return;
113     }
114
115     renderEmptyState(expressionResults, " 正在识别", " 图片分析中，请稍候。");
116
117     if (image) {
118       image.remove();
119     }
120     if (canvas) {
121       canvas.remove();
122     }
123
124     image = await faceapi.bufferToImage(imageUpload.files[0]);
125     container.append(image);
126
127     canvas = faceapi.createCanvasFromMedia(image);
128     container.append(canvas);
129
130     const displaySize = { width: image.width, height: image.height };
131     faceapi.matchDimensions(canvas, displaySize);
132
133     const detections = await faceapi
134       .detectAllFaces(image)
135       .withFaceLandmarks()
136       .withFaceDescriptors()
137       .withFaceExpressions();
138
```

```
139     const resizedDetections = faceapi.resizeResults(detections, displaySize);
140     faceapi.draw.drawDetections(canvas, resizedDetections);
141     faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
142     renderExpressionResults(resizedDetections);
143
144     analysisStage.classList.add("stage-ready");
145   });
146 }
```

年龄与性别估计脚本

年龄与性别估计脚本与表情识别脚本结构相近，但加载的任务模型不同。它除了检测、关键点和描述子模型外，还加载，用于输出估计年龄、性别标签和性别分类置信度。

```
1  const imageUpload = document.getElementById("upload");
2  const analysisStage = document.getElementById("analysis-stage");
3  const estimationResults = document.getElementById("estimation-results");
4  const MODEL_URL = "./models";
5
6  const GENDER_LABELS = {
7    male: " 男性",
8    female: " 女性",
9  };
10
11  Promise.all([
12    faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
13    faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
14    faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
15    faceapi.nets.ageGenderNet.loadFromUri(MODEL_URL),
16    faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
17  ]).then(start);
```

结果渲染部分把模型返回的 和 映射为中文展示文本，并将年龄估计值四舍五入后写入结果卡片。性别置信度继续用百分比和进度条表示，使侧边栏结果能够与右侧检测框形成对照。

```
38  function renderEstimationResults(detections) {
39    const sortedDetections = getSortedDetections(detections);
40  }
```

```
41 if (!sortedDetections.length) {
42   renderEmptyState(
43     estimationResults,
44     " 未检测到人脸",
45     " 请尝试上传清晰、无遮挡的人像照片。"
46   );
47   return;
48 }
49
50 estimationResults.innerHTML = sortedDetections
51   .map((detection, index) => {
52     const gender = detection.gender || "male";
53     const genderLabel = GENDER_LABELS[gender] || gender;
54     const confidence = detection.genderProbability || 0;
55
56     return `
57       <article class="result-card">
58         <div class="result-card__header">
59           <span class="result-card__title"> 人脸 ${index + 1}</span>
60           <span class="status-pill">${genderLabel}</span>
61         </div>
62         <div class="result-detail-grid">
63           <div class="result-detail">
64             <span class="result-detail__label"> 年龄估计 </span>
65             <span class="result-detail__value">${Math.round(
66               detection.age
67             )} 岁 </span>
68           </div>
69           <div class="result-detail">
70             <span class="result-detail__label"> 性别判断 </span>
71             <span class="result-detail__value">${genderLabel}</span>
72           </div>
73         </div>
74         <ul class="result-list">
75           <li class="result-metric">
76             <div class="result-metric__row">
77               <span class="result-metric__label"> 性别置信度 </span>
78               <span class="result-metric__value">${formatPercent(
79                 confidence
80               )}</span>
81             </div>
82             <div class="result-bar"><span style="width: ${Math.round(
83               confidence * 100
```

```
84         })%></span></div>
85     </li>
86 </ul>
87 </article>
88 `;
89 })
90 .join("");
91 }
```

图片推理流程仍然从上传事件开始。与表情识别不同的是，链式调用最后使用，并在画布上额外通过 绘制包含年龄和性别的检测框标签。

```
93 async function start() {
94     const container = document.createElement("div");
95     container.className = "analysis-stage__media";
96     analysisStage.append(container);
97
98     let image;
99     let canvas;
100
101     imageUpload.addEventListener("change", async () => {
102         if (!imageUpload.files.length) {
103             return;
104         }
105
106         renderEmptyState(estimationResults, " 正在识别", " 图片分析中，请稍候。");
107
108         if (image) {
109             image.remove();
110         }
111         if (canvas) {
112             canvas.remove();
113         }
114
115         image = await faceapi.bufferToImage(imageUpload.files[0]);
116         container.append(image);
117
118         canvas = faceapi.createCanvasFromMedia(image);
119         container.append(canvas);
120
121         const displaySize = { width: image.width, height: image.height };
122         faceapi.matchDimensions(canvas, displaySize);
```

```
123
124     const detections = await faceapi
125         .detectAllFaces(image)
126         .withFaceLandmarks()
127         .withFaceDescriptors()
128         .withAgeAndGender();
129
130     const resizedDetections = faceapi.resizeResults(detections, displaySize);
131     faceapi.draw.drawDetections(canvas, resizedDetections);
132
133     resizedDetections.forEach((detection, index) => {
134         const box = resizedDetections[index].detection.box;
135         const drawBox = new faceapi.draw.DrawBox(box, {
136             label: `${Math.round(detection.age)} years old ${detection.gender}`,
137         });
138         drawBox.draw(canvas);
139     });
140
141     renderEstimationResults(resizedDetections);
142
143     analysisStage.classList.add("stage-ready");
144 });
145 }
```

摄像头实时追踪脚本

摄像头脚本在页面初始化时保存视频元素、追踪区域、状态区和结果区引用，并维护覆盖画布、定时器和结果刷新时间等状态。模型加载完成后进入，为后续实时识别做准备。

```
1     const video = document.getElementById("video");
2     const trackingStage = document.getElementById("tracking-stage");
3     const webcamStatus = document.getElementById("webcam-status");
4     const webcamResults = document.getElementById("webcam-results");
5     const MODEL_URL = "./models";
6     let canvas;
7     let detectionInterval;
8     let lastResultsMarkup = "";
9     let lastResultsRenderAt = 0;
10
11     const EXPRESSION_LABELS = {
12         neutral: "中性",
```



```
13   happy: "高兴",
14   sad: "难过",
15   angry: "生气",
16   fearful: "害怕",
17   disgusted: "厌恶",
18   surprised: "惊讶",
19 };
20 const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
21 const SIDEBAR_REFRESH_MS = 280;
22
23 Promise.all([
24   faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
25   faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
26   faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
27   faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
28 ]).then(startVideo);
29
30 function getDisplaySize() {
31   return {
32     width: Math.max(trackingStage.clientWidth, 1),
33     height: Math.max(trackingStage.clientHeight, 1),
34   };
35 }
36
37 function syncCanvasSize() {
38   if (!canvas) {
39     return getDisplaySize();
40   }
41
42   const displaySize = getDisplaySize();
43
44   if (canvas.width !== displaySize.width || canvas.height !== displaySize.height) {
45     faceapi.matchDimensions(canvas, displaySize);
46   }
47
48   return displaySize;
49 }
```

实时页面需要频繁更新侧边栏，因此脚本把结果 HTML 的生成和渲染拆开处理。渲染函数会比较本次结果与上次结果，并使用 `requestAnimationFrame` 控制刷新间隔，避免每 100 ms 都重复写入 DOM。

```
87 function getTopExpression(expressions) {
88   return EXPRESSION_ORDER.reduce(
89     (currentTop, key) => {
90       const score = expressions[key] || 0;
91       return score > currentTop.score ? { key, score } : currentTop;
92     },
93     { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
94   );
95 }
96
97 function buildResultsMarkup(detections) {
98   const sortedDetections = getSortedDetections(detections);
99
100   if (!sortedDetections.length) {
101     return `
102     <div class="result-empty">
103       <strong> 未检测到人脸 </strong>
104       <span> 当前画面中还没有检测到清晰人脸。</span>
105     </div>
106   `;
107   }
108
109   return sortedDetections
110     .map((detection, index) => {
111     const topExpression = getTopExpression(detection.expressions);
112     const metrics = EXPRESSION_ORDER.map((key) => {
113       const score = detection.expressions[key] || 0;
114
115       return `
116       <li class="result-metric">
117         <div class="result-metric__row">
118           <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
119           <span class="result-metric__value">${formatPercent(score)}</span>
120         </div>
121         <div class="result-bar"><span style="width: ${Math.round(
122           score * 100
123         )}%></span></div>
124       </li>
125     `;
126     }).join("");
127
128   return `
```

```

129     <article class="result-card">
130         <div class="result-card__header">
131             <span class="result-card__title"> 人脸 ${index + 1}</span>
132             <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
133         </div>
134         <p class="result-card__highlight">
135             主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
136             (${formatPercent(topExpression.score)})
137         </p>
138         <ul class="result-list">${metrics}</ul>
139     </article>
140     `;
141 })
142 .join("");
143 }
144
145 function renderWebcamResults(detections, force = false) {
146     const markup = buildResultsMarkup(detections);
147     const now = Date.now();
148
149     if (!force && markup === lastResultsMarkup) {
150         return;
151     }
152
153     if (!force && now - lastResultsRenderAt < SIDEBAR_REFRESH_MS) {
154         return;
155     }
156
157     webcamResults.innerHTML = markup;
158     lastResultsMarkup = markup;
159     lastResultsRenderAt = now;
160 }

```

摄像头启动函数兼容了现代 和旧版浏览器前缀接口。脚本会先显示等待授权状态，在用户允许访问摄像头后把视频流写入；如果浏览器不支持或授权失败，则更新状态区和结果区提示。

```

162 function startVideo() {
163     renderStatus(" 等待授权", " 未连接", " 允许浏览器访问摄像头后开始实时识别。");
164     renderEmptyResults(
165         " 等待识别",
166         " 摄像头开始工作后，这里会同步显示表情识别结果副本。",

```

```
167     true
168   );
169
170   const legacyGetUserMedia =
171     navigator.getUserMedia ||
172     navigator.webkitGetUserMedia ||
173     navigator.mozGetUserMedia;
174
175   const getUserMedia =
176     navigator.mediaDevices && navigator.mediaDevices.getUserMedia
177       ? (constraints) => navigator.mediaDevices.getUserMedia(constraints)
178       : legacyGetUserMedia
179       ? (constraints) =>
180         new Promise((resolve, reject) => {
181           legacyGetUserMedia.call(navigator, constraints, resolve, reject);
182         })
183       : null;
184
185   if (!getUserMedia) {
186     console.error("Camera API is not supported in this browser.");
187     renderStatus(" 摄像头不可用", " 不支持", " 当前浏览器不支持摄像头访问接口。");
188     renderEmptyResults(
189       " 无法开始识别",
190       " 请更换支持摄像头访问的浏览器后重试。",
191       true
192     );
193     return;
194   }
195
196   renderStatus(" 正在连接", " 连接中", " 正在请求摄像头权限, 请稍候。");
197
198   getUserMedia({ video: {} })
199     .then((stream) => {
200       video.srcObject = stream;
201       renderStatus(" 摄像头已连接", " 已连接", " 画面已连接, 等待浏览器开始播放视频流。");
202     })
203     .catch((error) => {
204       console.error(error);
205       renderStatus(
206         " 访问失败",
207         " 不可用",
208         " 未能获取摄像头权限, 或者当前设备不可用。"
209       );
210     });
```

```
210     renderEmptyResults(  
211         " 无法开始识别",  
212         " 请检查浏览器授权和摄像头设备后重试。",  
213         true  
214     );  
215 });  
216 }
```

当视频开始播放时，脚本创建覆盖画布并启动定时器。每轮定时任务都会同步画布尺寸，对当前视频帧执行人脸检测、关键点定位和表情识别，再清空上一帧标注并重绘最新检测框、关键点和表情标签。视频暂停或结束时，脚本及时清除定时器，避免后台继续执行无意义的推理。

```
218 video.addEventListener("play", () => {  
219     if (!canvas) {  
220         canvas = faceapi.createCanvasFromMedia(video);  
221         trackingStage.append(canvas);  
222     }  
223  
224     syncCanvasSize();  
225     trackingStage.classList.add("stage-live");  
226     renderStatus(" 正在识别", " 识别中", " 已连接摄像头，正在同步更新实时识别结果。");  
227  
228     window.clearInterval(detectionInterval);  
229     detectionInterval = window.setInterval(async () => {  
230         const displaySize = syncCanvasSize();  
231         const detections = await faceapi  
232             .detectAllFaces(video, new faceapi.TinyFaceDetectorOptions())  
233             .withFaceLandmarks()  
234             .withFaceExpressions();  
235  
236         const resizedDetections = faceapi.resizeResults(detections, displaySize);  
237         canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);  
238         faceapi.draw.drawDetections(canvas, resizedDetections);  
239         faceapi.draw.drawFaceLandmarks(canvas, resizedDetections);  
240         faceapi.draw.drawFaceExpressions(canvas, resizedDetections);  
241         renderWebcamResults(resizedDetections);  
242     }, 100);  
243 });  
244  
245 window.addEventListener("resize", () => {
```

```
246     syncCanvasSize();
247   });
248
249   ["pause", "ended"].forEach((eventName) => {
250     video.addEventListener(eventName, () => {
251       window.clearInterval(detectionInterval);
252       renderStatus(" 识别已暂停", " 已暂停", " 摄像头画面已暂停, 左侧结果已停止更新。");
253       renderEmptyResults(
254         " 识别已暂停",
255         " 恢复摄像头画面后, 这里会重新同步显示识别结果。",
256         true
257       );
258     });
259   });
```

开源人脸识别模型训练代码

本项目网页端直接使用 `face-api.js` 的预训练权重, 并没有在浏览器项目中重新训练深度网络。为了说明人脸识别模型的训练思想, 本节引用已经克隆到 `code/dlib/` 目录中的 `dlib` 开源训练示例 `dnn_metric_learning_on_images_ex.cpp`。该示例展示了如何使用 训练人脸嵌入模型, 使同一身份的人脸向量距离更近, 不同身份的人脸向量距离更远。

示例开头说明了深度度量学习在人脸识别中的作用: 模型不直接把图像分类为固定身份, 而是学习一个向量空间, 再用距离或最近邻方法完成识别。

```
1  // The contents of this file are in the public domain. See
   ↪ LICENSE_FOR_EXAMPLE_PROGRAMS.txt
2  /*
3
4   This is an example illustrating the use of the deep learning tools from the
5   dlib C++ Library. In it, we will show how to use the loss_metric layer to do
6   metric learning on images.
7
8   The main reason you might want to use this kind of algorithm is because you
9   would like to use a k-nearest neighbor classifier or similar algorithm, but
10  you don't know a good way to calculate the distance between two things. A
11  popular example would be face recognition. There are a whole lot of papers
12  that train some kind of deep metric learning algorithm that embeds face
13  images in some vector space where images of the same person are close to each
14  other and images of different people are far apart. Then in that vector
   space it's very easy to do face recognition with some kind of k-nearest
```

```

15     neighbor classifier.
16
17     In this example we will use a version of the ResNet network from the
18     dnn_imagenet_ex.cpp example to learn to map images into some vector space where
19     pictures of the same person are close and pictures of different people are far
20     apart.
21
22     You might want to read the simpler introduction to the deep metric learning
23     API, dnn_metric_learning_ex.cpp, before reading this example. You should
24     also have read the examples that introduce the dlib DNN API before

```

训练数据按身份目录组织。顶层目录下的每个子目录表示一个身份，子目录内保存该身份的多张图片。加载函数遍历这些子目录，形成按身份分组的图片列表，为后续 mini-batch 采样做准备。

```

35
36 // -----
37 ↪ ---
38
39 // We will need to create some functions for loading data. This program will
40 // expect to be given a directory structured as follows:
41 //     top_level_directory/
42 //         person1/
43 //             image1.jpg
44 //             image2.jpg
45 //             image3.jpg
46 //         person2/
47 //             image4.jpg
48 //             image5.jpg
49 //             image6.jpg
50 //         person3/
51 //             image7.jpg
52 //             image8.jpg
53 //             image9.jpg
54 //
55 // The specific folder and image names don't matter, nor does the number of folders or
56 // images. What does matter is that there is a top level folder, which contains
57 // subfolders, and each subfolder contains images of a single person.
58
59 // This function spiders the top level directory and obtains a list of all the
60 // image files.
61
62 std::vector<std::vector<string>> load_objects_list (

```

```

61     const string& dir
62 )
63 {
64     std::vector<std::vector<string>> objects;
65     for (auto subdir : directory(dir).get_dirs())
66     {
67         std::vector<string> imgs;
68         for (auto img : subdir.get_files())
69             imgs.push_back(img);
70
71         if (imgs.size() != 0)
72             objects.push_back(imgs);
73     }
74     return objects;
75 }
76
77 // This function takes the output of load_objects_list() as input and randomly
78 // selects images for training. It should also be pointed out that it's really
79 // important that each mini-batch contain multiple images of each person. This
80 // is because the metric learning algorithm needs to consider pairs of images
81 // that should be close (i.e. images of the same person) as well as pairs of
82 // images that should be far apart (i.e. images of different people) during each
83 // training step.
84 void load_mini_batch (
85     const size_t num_people,      // how many different people to include
86     const size_t samples_per_id, // how many images per person to select.
87     dlib::rand& rnd,
88     const std::vector<std::vector<string>>& objs,
89     std::vector<matrix<rgb_pixel>>& images,
90     std::vector<unsigned long>& labels
91 )
92 {
93     images.clear();
94     labels.clear();
95     DLIB_CASSERT(num_people <= objs.size(), "The dataset doesn't have that many people in
    ↪ it.");

```

网络结构部分定义了训练网络和测试网络。训练网络使用带批归一化的 ResNet 主体，输出层为，损失层为；测试网络则把批归一化替换为固定仿射变换，便于推理阶段稳定使用。

```

175 using net_type = loss_metric<fc_no_bias<128,avg_pool_everything<

```



```

176         level0<
177         level1<
178         level2<
179         level3<
180         level4<
181         max_pool<3,3,2,2,relu<bn_con<con<32,7,7,2,2,
182         input_rgb_image
183         >>>>>>>>>>;
184
185 // testing network type (replaced batch normalization with fixed affine transforms)
186 using anet_type = loss_metric<fc_no_bias<128,avg_pool_everything<
187         alevel0<
188         alevel1<

```

训练阶段创建，设置学习率和同步文件，并通过多个加载线程持续产生 mini-batch。每个 mini-batch 包含多个身份以及每个身份的多张图片，使模型可以同时学习同一身份和不同身份之间的距离关系。训练结束后，示例把网络清理并序列化保存为 `metric_network_renset.dat`。

```

220     dnn_trainer<net_type> trainer(net, sgd(0.0001, 0.9));
221     trainer.set_learning_rate(0.1);
222     trainer.be_verbose();
223     trainer.set_synchronization_file("face_metric_sync", std::chrono::minutes(5));
224     // I've set this to something really small to make the example terminate
225     // sooner. But when you really want to train a good model you should set
226     // this to something like 10000 so training doesn't terminate too early.
227     trainer.set_iterations_without_progress_threshold(300);
228
229     // If you have a lot of data then it might not be reasonable to load it all
230     // into RAM. So you will need to be sure you are decompressing your images
231     // and loading them fast enough to keep the GPU occupied. I like to do this
232     // using the following coding pattern: create a bunch of threads that dump
233     // mini-batches into dlib::pipes.
234     dlib::pipe<std::vector<matrix<rgb_pixel>>> qimages(4);
235     dlib::pipe<std::vector<unsigned long>> qlabels(4);
236     auto data_loader = [&qimages, &qlabels, &objs](time_t seed)
237     {
238         dlib::rand rnd(time(0)+seed);
239         std::vector<matrix<rgb_pixel>> images;
240         std::vector<unsigned long> labels;
241         while(qimages.is_enabled())

```

```
242     {
243         try
244         {
245             load_mini_batch(5, 5, rnd, objs, images, labels);
246             qimages.enqueue(images);
247             qlabels.enqueue(labels);
248         }
249         catch(std::exception& e)
250         {
251             cout << "EXCEPTION IN LOADING DATA" << endl;
252             cout << e.what() << endl;
253         }
254     }
255 };
256 // Run the data_loader from 5 threads. You should set the number of threads
257 // relative to the number of CPU cores you have.
258 std::thread data_loader1([data_loader]() { data_loader(1); });
259 std::thread data_loader2([data_loader]() { data_loader(2); });
260 std::thread data_loader3([data_loader]() { data_loader(3); });
261 std::thread data_loader4([data_loader]() { data_loader(4); });
262 std::thread data_loader5([data_loader]() { data_loader(5); });
263
264
265 // Here we do the training. We keep passing mini-batches to the trainer until the
266 // learning rate has dropped low enough.
267 while(trainer.get_learning_rate() >= 1e-4)
268 {
269     qimages.dequeue(images);
270     qlabels.dequeue(labels);
271     trainer.train_one_step(images, labels);
272 }
273
274 // Wait for training threads to stop
275 trainer.get_net();
276 cout << "done training" << endl;
277
278 // Save the network to disk
279 net.clean();
280 serialize("metric_network_renset.dat") << net;
```

训练完成后，示例把图片送入测试网络得到嵌入向量，并根据向量距离判断两张图片是否属于同一身份。这个过程与浏览器项目中的 思路一致：先得到人脸描述

子，再依据距离阈值完成匹配。

```
300 // Normally you would use the non-batch-normalized version of the network to do
301 // testing, which is what we do here.
302 anet_type testing_net = net;
303
304 // Run all the images through the network to get their vector embeddings.
305 std::vector<matrix<float,0,1>> embedded = testing_net(images);
306
307 // Now, check if the embedding puts images with the same labels near each other and
308 // images with different labels far apart.
309 int num_right = 0;
310 int num_wrong = 0;
311 for (size_t i = 0; i < embedded.size(); ++i)
312 {
313     for (size_t j = i+1; j < embedded.size(); ++j)
314     {
315         if (labels[i] == labels[j])
316         {
317             // The loss_metric layer will cause images with the same label to be less
318             // than net.loss_details().get_distance_threshold() distance from each
319             // other. So we can use that distance value as our testing threshold.
320             if (length(embedded[i]-embedded[j]) <
321                 ⇨ testing_net.loss_details().get_distance_threshold())
322                 ++num_right;
323             else
324                 ++num_wrong;
325         }
326         else
327         {
328             if (length(embedded[i]-embedded[j]) >=
329                 ⇨ testing_net.loss_details().get_distance_threshold())
330                 ++num_right;
```